Trinity, in simple words

A readable guide to the uploaded Trinity README v1.13, focused on fundamentals and useful commands for a Post-Trainee LLM role.

The big idea

Trinity is tooling for creating and evaluating **challenging AI evaluation tasks**, including adversarial reinforcement-learning environments. It helps a team design tasks, run them, audit evidence, and remember what was learned. Its principle is: **difficulty is measured, never claimed.**

The three instruments

INSTRUMENT	JOB	WORKBENCH / REPORT / COMMAND

ENGRAM	Remembers learning, tracks reusable techniques (“levers”), and guides the others.	`.memory/`
		`DIRECTIVE.md`
		`/engram`

FORGE	Creates task bundles. A normal batch has 30 slots plus at most one anchor.	`.seed/`
		`EDICT.md`
		`/forge`

CRUCIBLE	Audits tasks and project evidence.	`.audit/`
		`VERDICT.md`
		`/crucible`

Contracts outrank the README

The authoritative instructions are `ENGRAM.md`, `FORGE.md`, and `CRUCIBLE.md`. If a contract conflicts with the README, follow the contract. The README is the map; contracts define each instrument’s rules.

Important separation: FORGE and CRUCIBLE do not communicate directly. Otherwise, authors could tune tasks to whatever the auditor accepts (reward hacking). ENGRAM is the controlled bridge through `FORGE_VIEW` and `CRUCIBLE_VIEW`; the auditor’s view hides the hardness contract.

From idea to evaluated task

Trinity separates human requirements, authoring, execution, auditing, and evidence-based learning.

STAGE	WHAT HAPPENS	WHY IT MATTERS
1. Human inputs	Requirements in `requirements/`; human-graded reference bundles in `touchstones/`.	Defines intended behaviour and expectations.
2. FORGE	Authors bundles in batches of 30 ordinary slots plus at most one anchor.	Creates structured tasks.
3. CRUCIBLE	Audits every sealed bundle and relevant project evidence.	Checks validity and evidence behind claims.
4. Harness	Executes tasks under declared conditions/budget and records outcomes.	Execution machinery, not a fourth instrument.
5. External signoff	A permitted external signer signs run outcomes and evidence.	Makes conclusions accountable to a frozen setup.
6. ENGRAM	Folds signed outcomes and design records into memory and publishes an updated epoch.	Future tasks use measured learning.

Anchors: calibration before scaling

Anchors are the first **3–5 calibration tasks** created from requirements and touchstones. They are audited, run, and signed. They calibrate the process but do not prove the whole population is difficult. The README requires three `ANCHORED` results across at least two axes before ordinary batches can escape the pilot-required hold.

Measured difficulty

A task that a current frontier model solves cleanly fails the intended evaluation. A declared budget is a bound, not a measurement. Passing a validity screen does not prove difficulty. Even a 0% pass rate may mean the grader or harness is broken, so validate those first.

Reports: `DIRECTIVE.md` **(ENGRAM direction),** `EDICT.md` **(FORGE work), and** `VERDICT.md` **(CRUCIBLE audit).** `TRACKING.md` **is a generated dashboard, not an independent verdict.**

Terms and status codes

These words recur throughout the README.

TERM	PLAIN MEANING
Contract	Authoritative Markdown work order for an instrument.
Instrument	One of the three agents: ENGRAM, FORGE, CRUCIBLE.
Workbench	Instrument-owned folder: `.memory/`, `.seed/`, or `.audit/`.
Harness	Execution machinery/configuration; not an instrument.
Oracle	Trusted answer/checking mechanism for outcomes; its integrity must also be tested.
Lever	Reusable technique for making a task harder; stale levers can expire.
Tier	Difficulty band: Baseline, Hard, or Frontier-defeat.
Pilot	External signed run against a frozen task and frozen solver registry.
Gate	Named human stop/signoff recorded using a SHA-256 digest.
Disposition	Gate outcome: SHIP, HOLD, or BLOCK.

The three main dispositions

STATUS	MEANING	WHAT TO DO
SHIP	Eligible to move forward under applicable rules.	Check its scope; not a universal quality guarantee.
HOLD	Wait: evidence, calibration, or approval is missing.	Resolve the documented reason.
BLOCK	Refuse: a required condition failed.	Read refusal code and recovery steps.

Variants worth recognising

`SHIP:INFERRED` — Batch-inferred against a committed sample bound; not fully measured evidence.

`SHIP:CONDITIONAL` — Signed, time-limited bypass lifted an eligible HOLD. Never lifts BLOCK or creates a measured claim.

`HOLD:PILOT_REQUIRED` — Pilot/calibration evidence is insufficient.

`HOLD:SUPPRESSED_MEASUREMENT` — Measurement is suppressed/uncovered, so status cannot rise above HOLD.

`BLOCK:INVALID_PILOT` — The executed population/pilot cannot be proven valid.

`HOLD:PAROLED` — Report remains on hold while a cured alert is under parole.

`PIPELINE_STOCK_FULL` — The stock lane has reached its configured capacity.

Fail closed: if evidence or authority is missing, do not assume success. A check proves only the specific thing it was designed to check.

Where things live—and why controls matter

The layout separates human inputs, working areas, evidence, and release lanes.

PATH	PURPOSE
`requirements/`	Human-authored task requirements and instructions.
`touchstones/`	Human-graded reference bundles.
`staging/`	Bundles waiting for audit or processing.
`samples/`	Bundles placed in sample lane after required checks.
`delivery/`	Delivery lane for eligible bundles.
`stock/`	Stock/overflow lane; capacity is bounded.
`harness/`	Execution extension/configuration, audited separately.
`.memory/ .seed/ .audit/`	Three instrument workbenches: memory, authoring, audit.
`.trinity/`	Project-level Trinity state and records.
`.sentinel/ .bypass/`	Security/refusal records and signed bypass records.

Security in plain English

CONTROL	WHAT IT MEANS
SHA-256 hash	Detects changed bytes; does not prove content is true or good.
Digital signature	Links signoff to a signing key and protects integrity; not proof of correctness by itself.
Role separation	Creator should not simply certify their own work; different roles sign different stages.
Git submodules/remotes	Separate component histories and permissions.
External verifier	Adds independent checks. README says external quality-controls runner is still absent/deferred.

Roles and refusals

`gate_producer` produces a gate artifact, `gate_approver` approves the gate, and `tantric` co-signs final SHIP. Do not infer permissions from rank names; consult `ACTORS.md` and `SIGNOFF.md`.

When a gate refuses an action, read its code and recovery instructions. Repeatedly rerunning a gate to make a finding disappear can itself count as a strike. Do not edit sealed bundles, ledgers, or `trinity/` to bypass a refusal.

Slash commands vs terminal commands

Slash commands run inside a supported coding-agent interface. Terminal commands run in a shell.

SLASH COMMAND	USE
`/engram`	Bootstrap/start ENGRAM's research and memory workflow.
`/forge operations`	Bootstrap FORGE for task authoring.
`/author` / `/anchor`	Author normal tasks / an anchor calibration task.
`/harness`	Work on the execution harness.
`/crucible operations`	Bootstrap CRUCIBLE for operations auditing.
`/audit` / `/inspect`	Audit project evidence / inspect harness scope.
`/fold`	Fold signed evidence into ENGRAM memory and publish updated standing.
`/maestro`	Orchestrate instruments in fixed order; not a fourth instrument.
`/crisp` / `/report`	Refresh or produce reports; do not automatically repeat audits or change disposition.
`/request`	Request final signoff; notification is not itself a signature.

Existing project setup

This short path assumes prerequisites and permissions are in place. Replace `<trinity remote>` with the actual repository remote.

```
git submodule add -b main <trinity remote> trinity
git config -f .gitmodules submodule.trinity.update merge
uv run --project ./trinity --no-dev ./trinity/tools/gate.py install ./
```

Then open the coding agent at the project root. Run each bootstrap separately and sign each gate when it stops:

```
/engram
/forge operations
/crucible operations
```

Useful terminal commands

```
uv run --project ./trinity ./trinity/tools/identity.py register ./ --login <your-login>
uv run --project ./.memory engram genesis --target ./projects/<project>
uv run --project ./trinity --no-dev ./trinity/tools/eta.py report
uv run --project ./trinity --no-dev ./trinity/tools/standalone.py run <path>
just --justfile tools/justfile ci
```

These show identity-registration syntax, create a new knowledge project, report ETA/progress, run a standalone dataset assay, and run CI checks from the Trinity repository. Full onboarding also involves signing/accepting the identity request and installing the gate with the correct signing key and signed commit.

Other tools and what to remember

Know each subsystem's purpose and what it cannot prove; you do not need to memorise every internal script.

COMPONENT	PLAIN EXPLANATION
GAUNTLET	Out-of-band gatekeeper that challenges artifacts. It has no workbench and does not replace human signoff.
MAESTRO	Orchestrates ENGRAM, FORGE, and CRUCIBLE in fixed order; has no independent workbench/state/report.
SENTINEL	Tracks security intent/refusals. Testimony is not automatically evidence; alerts can block progress.
ETA / TRACKING	Shows progress, waiting, flow, and rework. Operational reporting is not a quality guarantee.
Standalone assay	Checks supported datasets for schema/field issues, near-duplicates, split leakage, and contamination. Unknown formats are held; its verdict does not grant standing in a parent project.
CI / tests	Checks doctrine/integrity, refusal citations, linting, types, and tests. Passing CI does not replace pilot evidence.

Phases and signoff

The README describes staged work: research (R), scope (0), human SHA-256 signoff (0.5), scaffold (1), self-verification (2), adversarial validity review (3), external evidence barrier (4), and release evidence signoff (4.5). ENGRAM adds genesis, hardness, succession, and publication phases; CRUCIBLE has a referee phase. Each stage has a defined stop, and a human signs the gate before later stages rely on it.

Five rules to remember

- Human-owned inputs such as `requirements/` and `touchstones/` should not be silently rewritten by agents.
- Schema versions matter; incompatible versions should be refused rather than guessed into compatibility.
- A valid-looking task or supplier document is not proof of provenance or difficulty.
- Test the harness and oracle/grader; do not assume either is correct.
- Task difficulty can expire as models improve, so evidence and claims need revisiting.

20-second explanation: Trinity creates adversarial AI evaluation tasks, executes them under a controlled harness, audits the evidence, and feeds signed results back into memory. ENGRAM remembers, FORGE creates, CRUCIBLE audits. Their separation reduces reward hacking; human signoffs and fail-closed gates make the process accountable.

For your role: focus on the three instruments, end-to-end workflow, why difficulty must be measured, what SHIP/HOLD/BLOCK mean, and why hashes, signatures, and role separation matter. For exact procedures, the contracts and `ACTORS.md`/`SIGNOFF.md` take priority.